

Abstract Data Types

Last Updated : 28 Mar, 2025

An **Abstract Data Type (ADT)** is a conceptual model that defines a set of operations and behaviors for a data structure, **without specifying how these operations are implemented** or how data is organized in memory. The definition of ADT only mentions what **operations are to be performed** but not **how** these operations will be implemented. It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations. **It is called "abstract" because it provides an implementation-independent view.**

The process of providing only the essentials and hiding the details is known as [abstraction](#).

Features of ADT

Abstract data types (ADTs) are a way of encapsulating data and operations on that data into a single unit. Some of the key features of ADTs include:

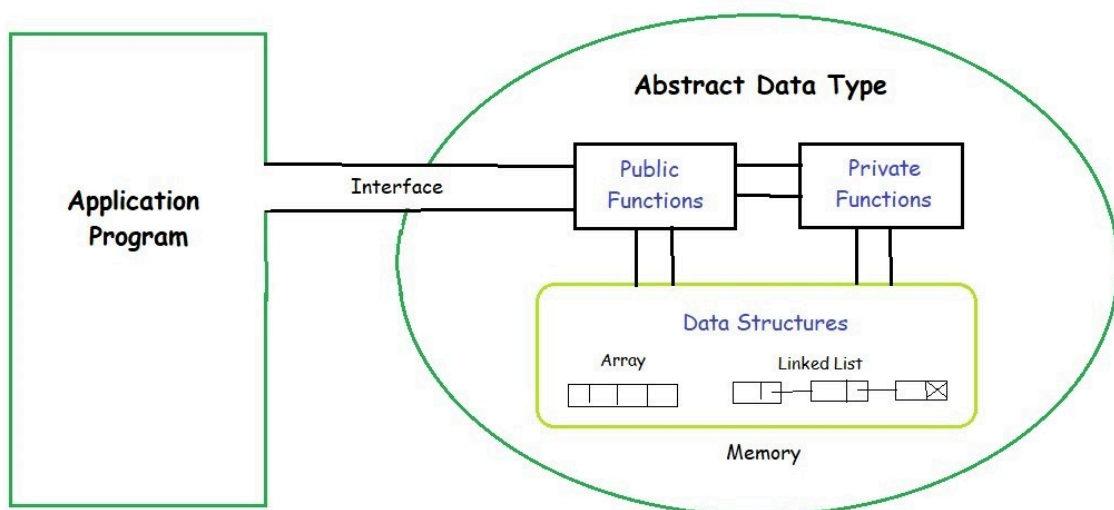
- **Abstraction:** The user does not need to know the implementation of the data structure only essentials are provided.
- **Better Conceptualization:** ADT gives us a better conceptualization of the real world.
- **Robust:** The program is robust and has the ability to catch errors.
- **Encapsulation:** ADTs hide the internal details of the data and provide a public interface for users to interact with the data. This allows for easier maintenance and modification of the data structure.
- **Data Abstraction:** ADTs provide a level of abstraction from the implementation details of the data. Users only need to know the

operations that can be performed on the data, not how those operations are implemented.

- **Data Structure Independence:** ADTs can be implemented using different data structures, such as arrays or linked lists, without affecting the functionality of the ADT.
- **Information Hiding:** ADTs can protect the integrity of the data by allowing access only to authorized users and operations. This helps prevent errors and misuse of the data.
- **Modularity:** ADTs can be combined with other ADTs to form larger, more complex data structures. This allows for greater flexibility and modularity in programming.

Overall, ADTs provide a powerful tool for organizing and manipulating data in a structured and efficient manner.

This image demonstrates how an Abstract Data Type (ADT) hides internal data structures (like arrays, linked lists) using public and private functions, exposing only a defined interface to the application program.



Why Use ADTs?

The key reasons to use ADTs in Java are listed below:

- **Encapsulation:** Hides complex implementation details behind a clean interface.

- **Reusability:** Allows different internal implementations (e.g., array or linked list) without changing external usage.
- **Modularity:** Simplifies maintenance and updates by separating logic.
- **Security:** Protects data by preventing direct access, minimizing bugs and unintended changes.

Example of Abstraction

For example, we use primitive values like int, float, and char with the understanding that these data types can operate and be performed on without any knowledge of their implementation details. ADTs operate similarly by defining **what operations are possible without detailing their implementation.**

Difference Between ADTs and UDTs

The table below demonstrates the difference between ADTs and UDTs.

Aspect	Abstract Data Types (ADTs)	User-Defined Data Types (UDTs)
Definition	Defines a class of objects and the operations that can be performed on them, along with their expected behavior (semantics), but without specifying implementation details.	A custom data type created by combining or extending existing primitive types, specifying both structure and operations.
Focus	What operations are allowed and how they behave, without dictating	How data is organized in memory and how

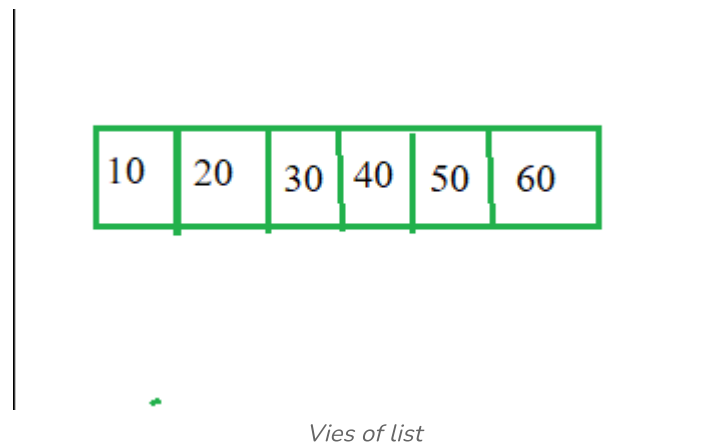
Aspect	Abstract Data Types (ADTs)	User-Defined Data Types (UDTs)
	how they are implemented.	operations are executed.
Purpose	Provides an abstract model to define data structures in a conceptual way.	Allows programmers to create concrete implementations of data structures using primitive types.
Implementation Details	Does not specify how operations are implemented or how data is structured.	Specifies how to create and organize data types to implement the structure.
Usage	Used to design and conceptualize data structures.	Used to implement data structures that realize the abstract concepts defined by ADTs.
Example	List ADT, Stack ADT, Queue ADT.	Structures, classes, enumerations, records.

Examples of ADTs

Now, let's understand three common ADT's: List ADT, Stack ADT, and Queue ADT.

1. List ADT

The List ADT (Abstract Data Type) is a sequential collection of elements that supports a set of operations **without specifying the internal implementation**. It provides an ordered way to store, access, and modify data.



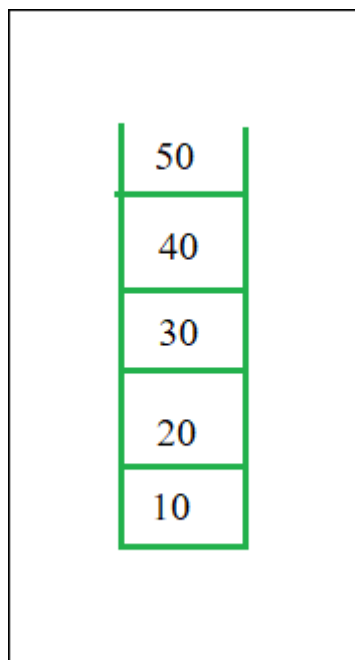
Operations:

The List ADT need to store the required data in the sequence and should have the following operations:

- **get():** Return an element from the list at any given position.
- **insert():** Insert an element at any position in the list.
- **remove():** Remove the first occurrence of any element from a non-empty list.
- **removeAt():** Remove the element at a specified location from a non-empty list.
- **replace():** Replace an element at any position with another element.
- **size():** Return the number of elements in the list.
- **isEmpty():** Return true if the list is empty; otherwise, return false.
- **isFull():** Return true if the list is full, otherwise, return false. Only applicable in fixed-size implementations (e.g., array-based lists).

2. Stack ADT

The Stack ADT is a linear data structure that follows the LIFO (Last In, First Out) principle. It allows elements to be added and removed only from one end, called the top of the stack.

*View of stack*

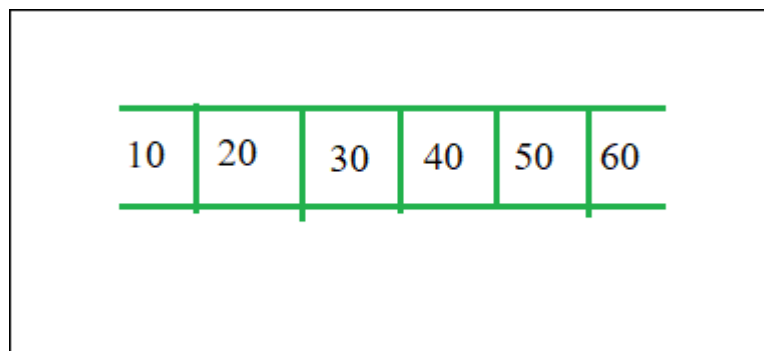
Operations:

In Stack ADT, the order of insertion and deletion should be according to the FILO or LIFO Principle. Elements are inserted and removed from the same end, called the top of the stack. It should also support the following operations:

- **push():** Insert an element at one end of the stack called the top.
- **pop():** Remove and return the element at the top of the stack, if it is not empty.
- **peek():** Return the element at the top of the stack without removing it, if the stack is not empty.
- **size():** Return the number of elements in the stack.
- **isEmpty():** Return true if the stack is empty; otherwise, return false.
- **isFull():** Return true if the stack is full; otherwise, return false. Only relevant for fixed-capacity stacks (e.g., array-based).

3. Queue ADT

The Queue ADT is a linear data structure that follows the FIFO (First In, First Out) principle. It allows elements to be inserted at one end (rear) and removed from the other end (front).

*View of Queue*

Operations:

The Queue ADT follows a design similar to the Stack ADT, but the order of insertion and deletion changes to FIFO. Elements are inserted at one end (called the rear) and removed from the other end (called the front). It should support the following operations:

- **enqueue():** Insert an element at the end of the queue.
- **dequeue():** Remove and return the first element of the queue, if the

DSA Tutorial Array Strings Linked List Stack Queue Tree Graph

Sign In

- **peek():** Return the element of the queue without removing it, if the queue is not empty.
- **size():** Return the number of elements in the queue.
- **isEmpty():** Return true if the queue is empty; otherwise, return false.

Advantages and Disadvantages of ADT

Abstract data types (ADTs) have several advantages and disadvantages that should be considered when deciding to use them in software development. Here are some of the main advantages and disadvantages of using ADTs:

Advantage:

The advantages are listed below:

- **Encapsulation:** ADTs provide a way to encapsulate data and operations into a single unit, making it easier to manage and modify the data structure.
- **Abstraction:** ADTs allow users to work with data structures without having to know the implementation details, which can simplify

programming and reduce errors.

- **Data Structure Independence:** ADTs can be implemented using different data structures, which can make it easier to adapt to changing needs and requirements.
- **Information Hiding:** ADTs can protect the integrity of data by controlling access and preventing unauthorized modifications.
- **Modularity:** ADTs can be combined with other ADTs to form more complex data structures, which can increase flexibility and modularity in programming.

Disadvantages:

The disadvantages are listed below:

- **Overhead:** Implementing ADTs can add overhead in terms of memory and processing, which can affect performance.
- **Complexity:** ADTs can be complex to implement, especially for large and complex data structures.
- **Learning Curve:** Using ADTs requires knowledge of their implementation and usage, which can take time and effort to learn.
- **Limited Flexibility:** Some ADTs may be limited in their functionality or may not be suitable for all types of data structures.
- **Cost:** Implementing ADTs may require additional resources and investment, which can increase the cost of development.

[Comment](#)[More info](#)

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem



Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Campus Training Program

Tutorials

Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
GATE
Software and Tools

Videos

Explore

POTD
Job-A-Thon
Community
Blogs
Nation Skill Up

Courses

IBM Certification
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Preparation Corner

DSA	Aptitude
Python	Puzzles
Java	GfG 160
C++	DSA 360
Web Development	System Design
Data Science	
CS Subjects	

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved